

Programming Language Constructs F# Intro

Peter Sestoft
Niels Hallenberg
2026-07-22



F# values, declarations and types

Expression

```
let res = 3+4;;
```

Declaration

```
let y = sqrt 2.0;;
```

```
let large = 10 < res;;
```

```
y > 0.0 && 1.0/y > 7.0;;
```

```
if 3 < 4 then 117 else 118;;
```

```
let rektor = "Mads " + "Tofte";;
```

- What types are `res`, `y`, `large` and `rektor`?
- What other types are there in F#?
- How compute the diagonal of a rectangle 3 by 5 m?

F# function definitions

```
let circleArea r = System.Math.PI * r * r;;  
  
let mul2 x = 2.0 * x;;
```

- A function that concatenates a string with itself?
- A function that finds the average of two floating-point numbers?

F# recursive function definitions

```
let rec fac (n : int) : int =  
    if n=0 then 1  
    else n * fac(n-1) ;;
```

- A function to compute the integer logarithm?
(That is, the number of times the integer can be halved before it is less than or equal to 1)
- A function that concatenates a string with itself n times?

F# type constraints

```
let isLarge x = 10 < x;;  
val isLarge : int -> bool
```

```
let isLarge (x : float) : bool = 10.0 < x;;  
val isLarge : float -> bool
```

- What if we give a wrong type constraint?

F# pattern matching

```
let rec fac n =  
    match n with  
        | 0 -> 1  
        | _ -> n * fac(n-1) ;;
```

- A pattern can be
 - a variable
 - a constant
 - a wildcard (`_`)
 - a constructor application `x :: xr`
 - a list `[]` or `[x]` or `x::y::xr`
 - a tuple `(x, y)` or `(2, 29)` or `([], x::xr)`

F# pairs and tuples

```
let p = (2, 3) ;;  
let w = (2, true, 3.4, "blah") ;;  
  
let add (x, y) = x + y ;;
```

```
let noon = (12, 0) ;;  
let talk = (15, 15) ;;  
  
let earlier ((h1, m1), (h2, m2)) =  
    h1 < h2 || (h1 = h2 && m1 < m2) ;;
```

F# lists

```
let x1 = [7; 9; 13];;  
let x2 = 7 :: 9 :: 13 :: [];;  
let equal = (x1 = x2);;
```

int list

string list

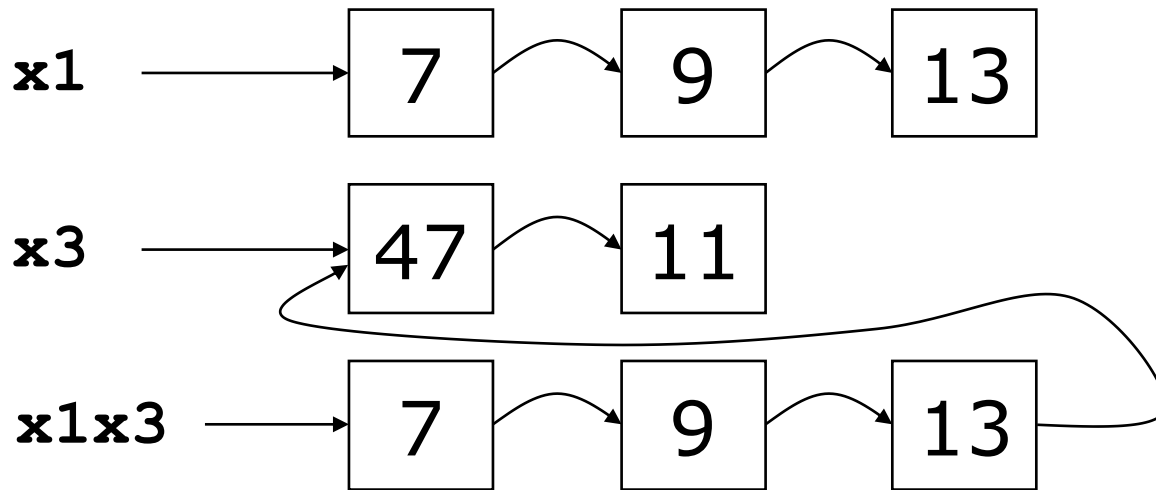
```
let ss = ["Dear"; title; name; "you have ..."];;  
let junkmail2 = String.concat " " ss;;
```

- A list of truth values?
- A list of pairs of name and age?
- What type would that list have?
- A function `makelist : int -> int list`, so that `makelist n = [n; n-1; ...; 1]`?

List append (@)

```
let x1 = [7; 9; 13];;  
let x3 = [47; 11];;  
  
let x1x3 = x1 @ x3;;
```

Result is
[7; 9; 13; 47; 11]



- F# data (lists, pairs, ...) are *immutable*
- This makes list sharing *unobservable*

F# defining functions on lists

```
let rec sum xs =  
    match xs with  
    | []      -> 0  
    | x::xr   -> x + sum xr;;
```

- Compute the length of a list?
- Compute the average of a list?
- Find maximum of a list of positive numbers?

F# record types and records

```
type phonerec =  
    { name : string; phone : int };;  
let x =  
    { name = "Kasper"; phone = 5170 };;
```

```
x.name;;  
x.phone;;
```

- A record type for course information: title, teacher, semester?

F# exceptions: raise and catch

```
exception IllegalHour;;  
let mins h =  
    if h < 0 || h > 23 then raise IllegalHour  
    else h * 60;;
```

```
try (mins 25)  
with IllegalHour -> -1;;
```

failwith raises System.Exception

```
let mins h =  
    if h < 0 || h > 23 then failwith "Illegal hour"  
    else h * 60;;
```

```
let mins h =  
    if h < 0 || h > 23 then  
        failwithf "Illegal hour, h=%d" h  
    else  
        h * 60;;
```

Formatted
failwith

Like C
printf

```
mins 25;;  
[...] System.Exception: Illegal hour, h=25  
[...]
```

F# algebraic datatypes

- Algebraic datatype, discriminated union
- A person is either a teacher or a student:

```
type person =  
    | Student of string  
    | Teacher of string * int;;
```

```
let people = [Student "Niels"; Teacher("Peter", 5083)];;
```

```
let getphone person =  
    match person with  
        | Teacher(name, phone) -> phone  
        | Student name         -> failwith "no phone";;
```

- A type to represent weekdays?
- A type to represent vehicles (car, bike, bus)?
- How would you do person/Student/Teacher in Java/C#?

F# curried functions

```
let addp (x, y) = x + y;;  
let res1 = addp(17, 25) ;;
```

Type?

```
let addc x y = x + y;;  
let res2 = addc 17 25;;
```

Type?

- Function application is left associative:
`addc x y` means `(addc x) y`
- The function type arrow is right associative:
`int -> int -> int` means `int -> (int -> int)`
- What would `(int -> int) -> int` mean?

F# polymorphic functions

```
let rec len xs =  
  match xs with  
  | []      -> 0  
  | x::xr   -> 1 + len xr;;
```

The function
doesn't look at
the list elements

```
val len : 'a list -> int
```

... so the function
is polymorphic

```
len [7; 9; 13]  
len [true; true; false; true]  
len ["foo"; "bar"]  
len [("Peter", 50)]
```

... and works in
any type of list

- Same as a generic **method** in Java or C#

```
static int Count<T>(IEnumerable<T> xs) { ... }
```


F# polymorphic types

```
type 'a tree =  
    | Lf  
    | Br of 'a * 'a tree * 'a tree
```

The datatype has
same structure
regardless of
node value type

```
Br(42, Lf, Lf)  
Br("quoi?", Lf, Lf)  
Br(("Peter", 50), Lf, Lf)
```

What type
instances here?

- Same as a generic **type** in Java or C#:

```
class ArrayList<T> { ... }  
interface IEnumerable<T> { ... }  
struct Pair<T,U> { ... }  
delegate R Func<A,R>(A x);
```

Kinds of polymorphism

- *Parametric polymorphism*, as in ML, F#, Java and C#:
 - The type variable 'a' stands for an arbitrary type
 - A parametric polymorphic function works the same way regardless what the type variable stands for
- *Bounded parametric polymorphism*, as in Java, C#:
 - The type variable T stands for a type with certain properties
 - For instance, a List<T> is printable if all its elements are:

```
class List<T> : IPrintable where T : IPrintable { ... }
```

- *Ad hoc polymorphism*, or *overloading*:
 - Java operator (+) works on int, double, String but not boolean
- *Virtual method calls* are sometimes said to be 'polymorphic'

```
Vehicle v = getMyVehicle();  
... v.getWeight() ...
```

May call getWeight()
on Bike, Car, Tank, ...

- A parametric polymorphic type is an assertion about a function
 - What terminating pure F# function has type 'a -> 'a ??
 - What terminating pure F# function has type 'a * 'b -> 'b * 'a ??

Polymorphic functions on polymorphic types

```
let rec preorder1 t =  
  match t with  
  | Lf          -> []  
  | Br(v, t1, t2) -> v :: preorder1 t1 @ preorder1 t2
```

- Return the tree's node values in pre-order
 - first root, then left subtree, then right subtree
- Works on any type of tree
- What is the type of this function?

Accumulating parameters

- The append (@) operation may be slow
- A faster version of preorder, no append!

```
let rec preo t acc =  
  match t with  
  | Lf          -> acc  
  | Br(v, t1, t2) -> v :: preo t1 (preo t2 acc) ;;  
  
let preorder2 t = preo t [];;
```

Accumulating
parameter

$O(n)$ versus $O(n^2)$

Can be 1000 x faster
Try `#time;;` in F#

- Function `preorder2` is correct because:

`preo t acc = preorder1 t @ acc`

- and therefore:

`preorder2 t = preo t [] = preorder1 t`



Proof, by induction on the tree

- Case $t = \text{Lf}$:

preo Lf acc

$= \text{acc}$

$= [] @ \text{acc}$

$= \text{preorder1 Lf} @ \text{acc}$

Ikke pensum,
bare nyttigt...

- Case $t = \text{Br}(v, t1, t2)$:

$\text{preo}(\text{Br}(v, t1, t2)) \text{acc}$

$= v :: \text{preo } t1 (\text{preo } t2 \text{acc})$

$= v :: \text{preo } t1 (\text{preorder1 } t2 @ \text{acc})$

$= v :: \text{preorder1 } t1 @ (\text{preorder1 } t2 @ \text{acc})$

$= (v :: \text{preorder1 } t1 @ \text{preorder1 } t2) @ \text{acc}$

$= \text{preorder1}(\text{Br}(v, t1, t2)) @ \text{acc}$